

AI Study Companion

Comprehensive System Architecture, Data Pipelines & Technical Specifications

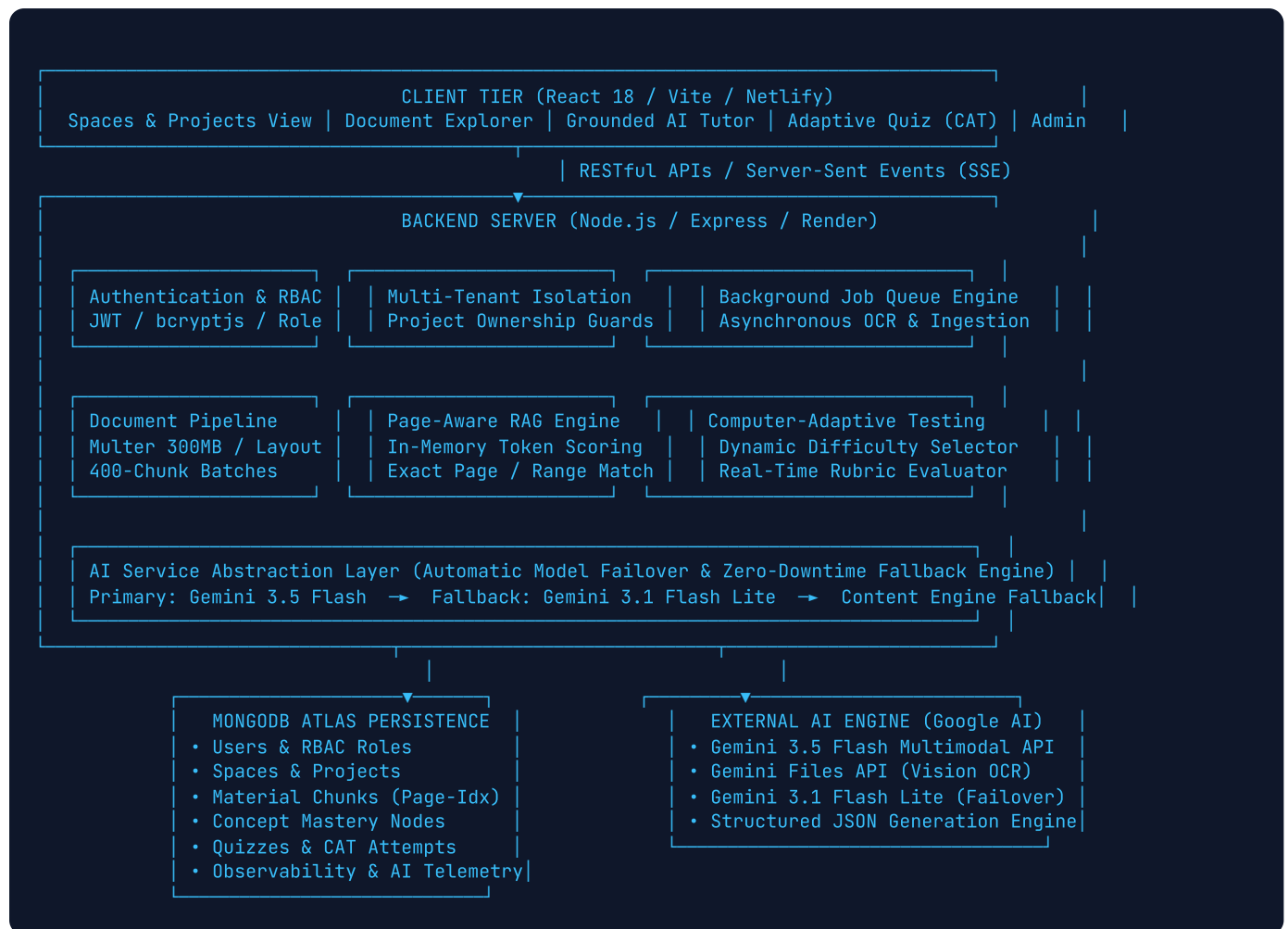
DOCUMENT VERSION	SYSTEM AUTHOR	DEPLOYMENT TARGET	CORE RUNTIME
v3.2 — Production Release	Engineering Team	Render (Backend) / Netlify (Frontend)	Node.js 18+ / React 18 / Gemini Flash

1. System Overview & Architectural Vision

The **AI Study Companion** is an enterprise-grade learning workspace engineered to facilitate personalized, mastery-based education over complex technical curricula, textbooks, and handwritten notes. Unlike typical superficial chatbots, the system operates as a closed-loop pedagogical engine:

- **Curriculum Structuring:** Organizes student learning into broad *Spaces* (e.g., Database Systems, Machine Learning) and targeted *Projects* (e.g., SQL Foundations, Gradient Descent Optimization).
- **High-Capacity Document Processing:** Ingests large textbooks and scanned slide decks up to 300MB and 500+ pages using coordinate-preserving layout extraction and batch insertion.
- **Multimodal Vision OCR:** Transcribes cursive handwriting, mathematical derivations, flowchart nodes, and SQL tables from camera/CamScanner scans using the Gemini Files API.
- **Strictly Grounded AI Tutor:** Answers student queries strictly using retrieved evidence chunks with exact page-level citations, refusing out-of-document trivia.
- **Computer-Adaptive Testing (CAT):** Dynamically adjusts quiz question difficulty question-by-question based on real-time performance.
- **Mastery Tracking & Observability:** Computes real-time concept mastery maps, actionable remedial recommendations, and token/latency telemetry for administrative oversight.

2. High-Level System Architecture Diagram



3. Core Architectural Subsystems

3.1 Multi-Tenant Hierarchy & RBAC Security Layer

The platform enforces complete tenant isolation across users. Data access follows a strict three-tier hierarchy: `User → Space → Project → Materials / Quizzes / Conversations`.

- **Isolation Middleware:** The `checkProjectAccess` middleware intercepts every project-scoped request. It resolves the project ID against the authenticated JWT user ID. Cross-tenant access attempts immediately terminate with HTTP 403 Forbidden.
- **Role-Based Access Control (RBAC):** Two strict roles exist: `user` (Learner) and `admin` (Instructor/Administrator). Administrators alone possess permission to access `/api/admin/*` observability dashboards, token expenditure logs, and platform latency metrics.
- **Idempotent Debounce Guards:** Space and project creation controllers enforce a 15-second debounce window preventing duplicate creation from rapid double-clicks.

3.2 High-Capacity Document Processing & Chunking Pipeline

Real-world university curricula involve dense slide decks, conference publications, and scanned textbooks. The ingestion pipeline supports files up to **300MB** and **500+ pages**:

- **Multer Storage Engine:** Configured with 300MB limits, memory-efficient streaming to disk, and specialized error traps preventing unhandled exceptions.
- **Coordinate-Preserving Layout Engine:** Uses `pdf-parse` stream hooks to preserve indentation, tabular relationships, headers, and code snippets.
- **400-Chunk Batch Ingestion:** To prevent MongoDB BSON document size limits (16MB) and network connection pool exhaustion during massive 500-page ingestions, chunks are partitioned and written in batches of 400 documents.
- **Scanner Artifact Filtration:** Automatically strips CamScanner watermarks (`"Scanned by CamScanner"` , `"Adobe Scan"` , isolated digit stamps) preventing skewed text density metrics.

3.3 Multimodal Vision OCR Engine for Handwritten Notes

When a student uploads camera photos or scanned handwritten notebooks (e.g., student lecture notes), standard text extraction produces empty or gibberish streams. The system deploys an automated Multimodal OCR sub-pipeline:

- **Watermark-Aware OCR Trigger:** Computes the real alphanumeric density per page. If text density falls below threshold, the document is flagged for Vision OCR.
- **Gemini Files API Ingestion:** The PDF binary is securely uploaded directly to Google Gemini Files API, generating a persistent remote URI.
- **Batch Processing Strategy:** To prevent token output exhaustion and network timeouts, large documents are processed in consecutive 15-page batches.
- **Demarcated Verbatim Extraction:** Each page is demarcated using `=== Page X ===` headers, transcribing handwritten mathematical formulas, SQL syntax, flowchart labels, and bulleted lists verbatim.

3.4 Scalable Page-Aware RAG Engine

The Retrieval-Augmented Generation (RAG) engine operates in-memory with sub-60ms retrieval latency across 500 pages (1,000+ chunks), implementing four distinct query routing mechanisms:

Query Type	Sample Query	Retrieval & Routing Strategy
Exact Page Intent	"What is in page 24?"	Isolates chunks strictly where <code>pageNumber === 24</code> ; suppresses noise from other sections.
Page Range Intent	"Summarize pages 10 to 20"	Filters chunks strictly within <code>[startPage, endPage]</code> , ranking by token density.
Full Overview / Summary	"Explain core concepts from uploaded material"	Performs stratified uniform sampling from Page 1 to Page N, ensuring representation across all chapters.
Terminal Page Intent	"What is in the last page?"	Resolves dynamic <code>maxPage</code> for the target document (e.g., Page 24 References).
Concept Search	"How does Gradient Descent minimize loss?"	Scores chunks via token coverage ratio (75%), token density (25%), and section header match boost.

Strict Document Grounding & Hallucination Guardrail

The AI Tutor is strictly restricted to the uploaded study materials. If a user asks a question outside the uploaded document (e.g., asking about *"father of nation"* inside an SQL project), the system immediately detects zero evidence matches, sets `isUnsupported: true` , clears citations, and politely refuses rather than hallucinating external trivia.

3.5 Computer-Adaptive Testing (CAT) & Mastery Engine

The assessment system implements an Item Response Theory (IRT)-inspired dynamic evaluation loop. Rather than static questionnaires, tests evolve dynamically across 6 to 8 questions:

- **Step-by-Step Question Generation:** Question 1 establishes baseline proficiency. Upon submission, the engine evaluates the response and calculates the next difficulty:
 - Score ≥ 70%: Escalates to **Advanced / Hard** difficulty.
 - Score 50% – 69%: Stabilizes at **Intermediate / Medium** difficulty.
 - Score < 50%: De-escalates to **Beginner / Easy** difficulty.
- **Rubric-Based AI Grading:** Open-ended responses are graded across accuracy, completeness, and clarity (0–100), awarding partial credit and constructive remedial feedback.
- **Concept Mastery Decay & Growth:** Every answered question updates the corresponding concept node mastery score in the database, reflecting real-time competency.

3.6 Telemetry & Administrative Observability

Every AI interaction across the platform is logged in the `AIUsageLog` collection with microsecond timestamps:

- Tracks prompt tokens, completion tokens, execution latency (ms), model used, user ID, and HTTP status.
- Calculates estimated operational costs in USD based on model pricing tiers.
- Surfaces error rates, peak latency percentiles, and active background job queues on the Admin Console.

4. Database Schema Specifications

Collection	Primary Fields	Indexes & Purpose
users	_id, name, email, password, role, avatarColor, preferences	Unique index on email .RBAC authentication and profile storage.
spaces	_id, title, description, color, userId, createdAt	Compound index on { userId: 1, createdAt: -1 } . Top-level subject domains.
projects	_id, spaceId, userId, title, learningGoal, progress, stats	Index on spaceId, userId . Focused learning workspaces.
materials	_id, projectId, originalName, fileType, pageCount, chunksCount, status	Index on projectId, status . Metadata for uploaded textbooks and notes.
materialchunks	_id, materialId, projectId, pageNumber, chunkIndex, content, keywords, tokenCount	Compound indexes on { projectId: 1, pageNumber: 1 } and { projectId: 1, chunkIndex: 1 } .
concepts	_id, projectId, title, description, mastery, prerequisites	Index on projectId . Dynamic knowledge graph nodes.
quizzes	_id, projectId, questions, totalTargetQuestions, sessionAnswers, isAdaptive	Index on projectId . Adaptive quiz configurations and state.
quizattempts	_id, quizId, userId, answers, score, evaluatedAnswers	Index on quizId, userId . Historical performance and rubric feedback.
aiusagelogs	_id, userId, projectId, feature, model, promptTokens, completionTokens, latencyMs, status	Compound index on { createdAt: -1, feature: 1 } . Observability telemetry.

5. Security, Scalability & Production Deployment

- **Frontend Deployment:** Hosted on **Netlify** with automated continuous integration from GitHub `main` , SPA routing rewrites via `netlify.toml` , and asset bundling.
- **Backend Deployment:** Hosted on **Render** running Node.js 18+, automated startup bootstrapping (`bootstrap.js`) ensuring admin account initialization, and health monitoring at `/health` .
- **Database:** **MongoDB Atlas** with replica sets, automated backups, and encrypted transit (TLS 1.3).
- **CORS & Security Headers:** Explicit CORS policies with credentials, JSON payload limits (10MB), and express rate limiting (180 requests/minute per IP).